Department of Computer Science and Engineering
University of Moratuwa

CS3713 - Image Processing



Assignment
JPEG compression

230415H Mudaliarachchi N.S.

20/10/2026

Contents

1 Introduction **2**

2 Compression Steps **2**

2.1 Color Space Conversion 2

2.2 Channel value normalization and chroma sub-sampling 3

2.3 Discrete Cosine Transform 4

2.4 Quantization 6

2.5 Run Length Encoding 8

2.6 Huffman encoding 9

3 Code Repository **11**

4 References **11**

1 Introduction

- JPEG is the common standard for containing lossy and compressed image data.
- Uncompressed Images require more space requirements. This leads to many problems like running out of memory.
- So, compressing images for sending or storing is the most critical part in today's applications.
- JPEG is a lossy compression format, dunning the compression of some original data in the image discarded to reduce image size.
- In JPEC compression, the algorithm itself throws away information in the original image that is not perceivable to human eyes.

2 Compression Steps

1. Color Space Conversion
2. Channel Value normalization and Chroma Sub-sampling
3. Discrete Cosine Transform
4. Quantization
5. Run Length Encoding.
6. Huffman (entropy) encoding.

2.1 Color Space Conversion

First we convert our image to YCrCb format. This is the first step of the image compression algorithm. Images can be represented in many color space forms. This process is a linear operation and that can be achieved by simple matrix multiplication.

```
def rgb_to_YCrCb(img):
    """

    This function is convert image RGB to YCrCb

    Args : Image
    Returns : Converted image

    Commonly used :

        Y = 0.299R + 0.587G + 0.114B

        Cr = 128 + 0.5R - 0.418688G - 0.081312B

        Cb = 128 - 0.168736R - 0.331264G + 0.5B

    """

    B = img[:, :, 0].astype(float)
```

```
    G = img[:, :, 1].astype(float)
    R = img[:, :, 2].astype(float)

    Y  = 0.299000 * R + 0.587000 * G + 0.114000 * B
    Cr = 128 + 0.500000 * R - 0.418688 * G - 0.081312 * B
    Cb = 128 - 0.168736 * R - 0.331264 * G + 0.500000 * B

    Y  = np.clip(np.round(Y),  0, 255).astype(np.uint8)
    Cr = np.clip(np.round(Cr), 0, 255).astype(np.uint8)
    Cb = np.clip(np.round(Cb), 0, 255).astype(np.uint8)

    ycrcb_img = np.stack((Y, Cr, Cb), axis=2)

    return ycrcb_img
```



Figure 1: Original 24bpp Image[3]



Figure 2: Converted Image in YCrCb

2.2 Channel value normalization and chroma sub-sampling

This is the second step of JPEG compression algorithm. In this step we have to normalize the channel values and down sampling of the chroma components(**Cr and Cb**). Normalization is done by subtracting 128 from every pixel value in the image. Algorithm only down sampling only to chroma components. Not for the luminance component(**Y**). Reason : Human Visual System is more sensitive to luminance than the chrominance. Therefor, by down sampling only the chroma components, the algorithm can reduce image size without affecting much of the information it provides to the viewer because a high proportion of that information is contained in only the luminance. This is called the **chroma sub-sampling** and this used in many type of image and video processing pipelines.

Following are the common Chroma Sub-sampling.

- 4:4:4 - Uncompressed image with no Chroma Sub-sampling
- 4:2:2 - Has half of the chroma 4:4:4 and reduces the size of an uncompressed image by one-third
- 4:2:2 - Has one quarter of the chroma of 4:4:4 and reduces the bandwidth of an uncompressed video signal by half compared to no chroma sub-sampling. [1]

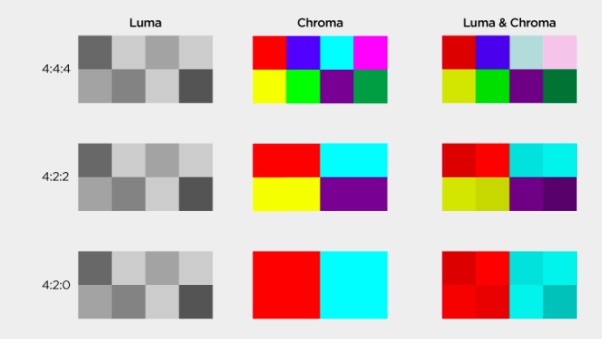


Figure 3: Chroma sub-sampling[1]

```python
def channel_norm_chroma_sub_sampeling(img):
    """
    Channel Normalization and the Chroma Sub Sampleing
    """

    Y = img[:,:, 0].astype(float)
    Cr = img[:,:,1].astype(float)
    Cb = img[:,:,2].astype(float)

    # 4:2:0 Chroma Subsampling
    Cb_sub = Cb[::2, ::2]
    Cr_sub = Cr[::2, ::2]

    # Channel normalization
    Y  = Y  - 128
    Cb_sub = Cb_sub - 128
    Cr_sub = Cr_sub - 128

    return Y, Cr_sub, Cb_sub

```

2.3 Discrete Cosine Transform

We use one special characteristic of human vision system, that can be exploited to reduce image size without affecting the image quality much is the frequency-dependent contracts sensitivity characteristic.

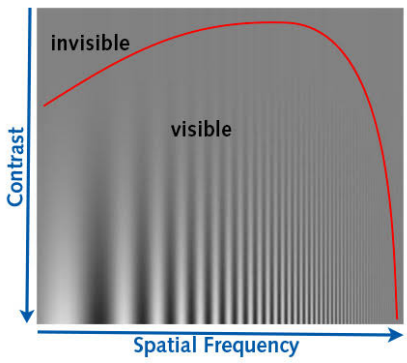


Figure 4: Frequency-dependent contracts sensitivity example

According to the Figure 4, Spatial frequency is increasing left to right, while contrast is decreasing bottom to top.

Due to **frequency-dependent contracts sensitivity** the bars under curve is more visible than the rest. According to the figure 2, **Low Spatial Frequencies** and **High Contrast clearly visible** to human eye while, **High Spatial Frequencies** and **Low Contrast barely visible**.

So the JPEG compression algorithm gives more room to compress the less frequencies(Barely visible regions). The algorithm does that by dividing the image into 8x8 blocks and quantize them in a frequency domain representation.

```
def dct_8x8(block):
    """
    Compute the 2D DCT for one 8x8 level-shifted block.
    """
    block = np.asarray(block, dtype=np.float64)

    result = np.zeros((8, 8), dtype=np.float64)

    for u in range(8):
        for v in range(8):
            total = 0.0
            for x in range(8):
                for y in range(8):
                    total += (
                        block[x, y] * np.cos((2 * x + 1) * u * np.pi / 16)* np.cos((2 * y + 1) * v * np.pi / 16))
            cu = 1 / np.sqrt(2) if u == 0 else 1.0
            cv = 1 / np.sqrt(2) if v == 0 else 1.0

            result[u, v] = 0.25 * cu * cv * total
    return result

def dct_channel(channel):
    """
    Apply 8x8 DCT to every block in a 2D channel.
    The channel dimensions must be multiples of 8.
    """
    channel = np.asarray(channel, dtype=np.float64)
    height, width = channel.shape

    transformed = np.zeros_like(channel)

    for i in range(0, height, 8):
        for j in range(0, width, 8):
            block = channel[i:i+8, j:j+8]
            transformed[i:i+8, j:j+8] = dct_8x8(block)

    return transformed
```

In the function $\text{dct}_{8\times8}$ I have implemented the DTC conversion for each 8*8 blocks. The DCT Equation represents in the below equation.

$$F(u,v) = \frac{1}{4}\,C(u)\,C(v)\sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y)\,\cos\left(\frac{(2x+1)u\pi}{16}\right)\cos\left(\frac{(2y+1)v\pi}{16}\right) \qquad (1)$$

where

$$C(k) = \begin{cases} \frac{1}{\sqrt{2}}, & k = 0 \\ 1, & k > 0 \end{cases} \tag{2}$$

- $f(x, y)$: pixel value in the level-shifted block.
- $F(u, v)$: DCT coefficient.
- $F(0, 0)$: DC coefficient, representing the block's average intensity up to a scaling factor.
- Other coefficients: AC coefficients, representing spatial frequencies.

The DCT is applied to each 8×8 pixel box and it compared with 64 different frequency patterns as shown in the following figure(Right hand side), where the spatial frequency increases from left to right and top to bottom.

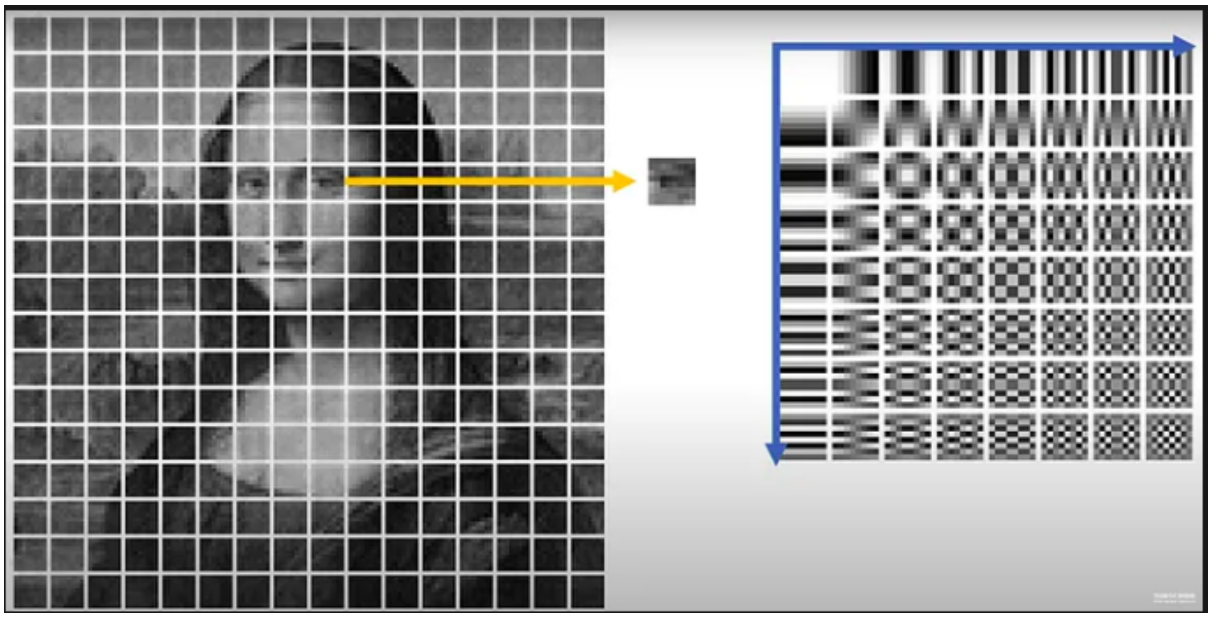


Figure 5: DCT Exampled Explained in the [2]

So the previous 8×8 represents the brightness and the DCT transformed 8×8 represents the particular frequency component.

2.4 Quantization

The next step to Quantize where the frequencies that are less visible to the human eye(The Right Conner of the image show in Figure 6).

Quantization is done by dividing frequency values by constants and get the integer value.To achieve this, the frequency components that are less sensitive to the human visual system get divided by larger constants as compared to the ones that humans are more sensitive to. After diving the result gets rounded to the nearest integers and this makes most of the high-frequency component (spread in the bottom right-hand corner)

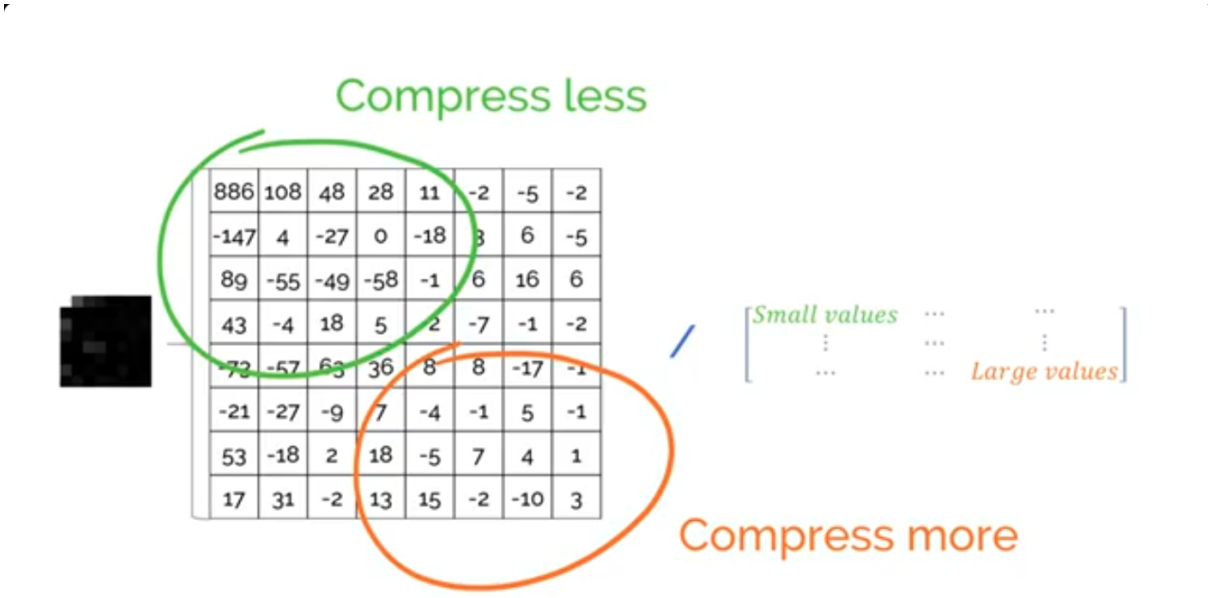


Figure 6: Exampled Discussed in the [2]

values become zero. This process achieves high compression rates but at the expense of image quality (lossy compression).

For the implementation I use the standard baseline JPEG quantization tables[4]. JPEG typically uses two 8×8 quantization tables, one for luminance (Y) and another for chrominance (Cb and Cr).

```
def Quntization(channel, channel_type):
    # Standard JPEG luminance quantization table
    QY = np.array([
        [16, 11, 10, 16, 24, 40, 51, 61],
        [12, 12, 14, 19, 26, 58, 60, 55],
        [14, 13, 16, 24, 40, 57, 69, 56],
        [14, 17, 22, 29, 51, 87, 80, 62],
        [18, 22, 37, 56, 68,109,103, 77],
        [24, 35, 55, 64, 81,104,113, 92],
        [49, 64, 78, 87,103,121,120,101],
        [72, 92, 95, 98,112,100,103, 99]
    ], dtype=np.float64)

    # Standard JPEG chrominance quantization table
    QC = np.array([
        [17, 18, 24, 47, 99, 99, 99, 99],
        [18, 21, 26, 66, 99, 99, 99, 99],
        [24, 26, 56, 99, 99, 99, 99, 99],
        [47, 66, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99],
        [99, 99, 99, 99, 99, 99, 99, 99]
    ], dtype=np.float64)

    height, width = channel.shape

```

```
    if channel_type == "Y":
        Q = QY
    elif channel_type == "C":
        Q = QC
    else:
        raise ValueError("channel_type must be 'Y' or 'C'")
    # repeats the block
    Q_full = np.tile(Q, (height // 8, width // 8))
    return np.rint(channel / Q_full)

```

2.5 Run Length Encoding

After the quantization process, all the higher frequencies became 0 in the 8×8 block. Instead of Store all data in each pixel, algorithm uses **run-length encoding** to achieve efficient storage of the same amount of data in a block (without losing data). In this step, each data item in an 8×8 block is rearranged in a zig-zag order from top left to bottom right. This allows the grouping of the zeros together and storing only one zero element followed by the number of times that zero element occur in the zigzag order.

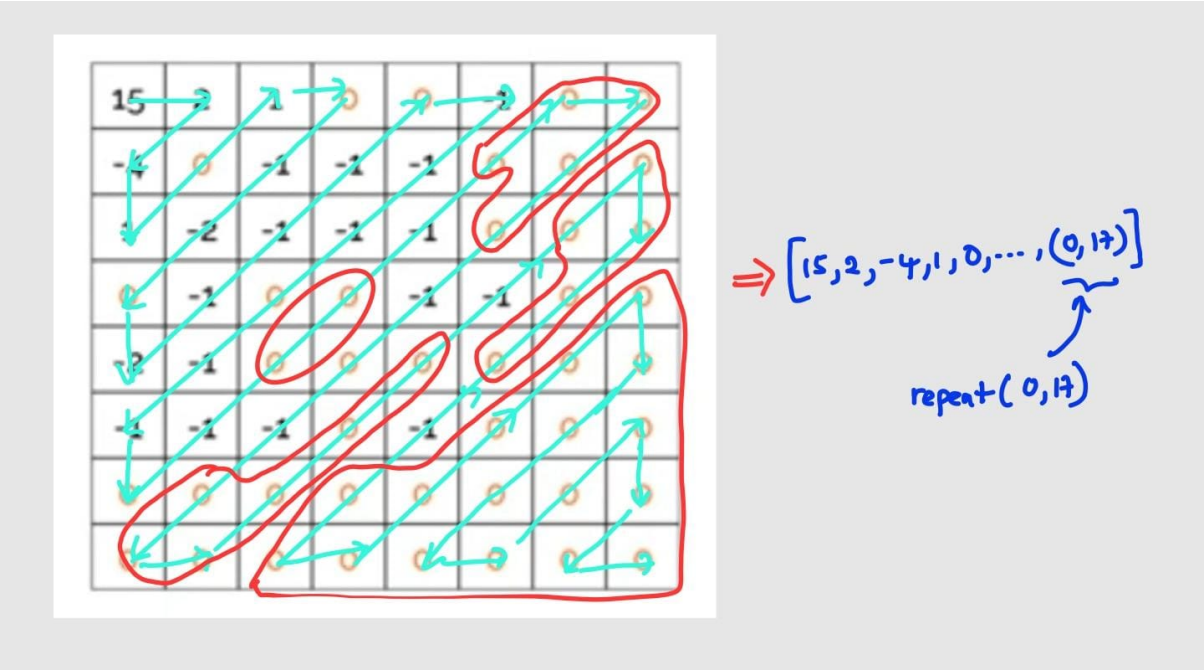


Figure 7: Run Length Encoding

```
def zigzag(block):
    # Zig-zag scan order for an 8x8 block (list of (row, col))
    ZIGZAG = sorted(((r, c) for r in range(8) for c in range(8)), key=
    lambda p: (p[0] + p[1], p[0] if (p[0] + p[1]) % 2 else p[1]))

    """
    Read an 8x8 block in zig-zag order -> 64 values (DC first).
    """
    return np.array([block[r, c] for r, c in ZIGZAG], dtype=np.int32)
```

```
9
10
11
12 def run_length_encoding(channel):
13     """
14     JPEG style RLE for a quantized channel.
15
16     DC : stored as the difference from the previous block's DC  (Top
    Left conner element)
17     AC : stored as (run_of_zeros, value) pairs (15, 0) = 16 zeros (ZRL)
    , (0, 0) = End Of Block (EOB)
18     """
19     height, width = channel.shape
20     dc_values = []
21     ac_values = []
22     prev_dc = 0
23
24     for i in range(0, height, 8):
25         for j in range(0, width, 8):
26             zz = zigzag(channel[i:i+8, j:j+8])
27
28             # DC differential
29             dc_values.append(int(zz[0]) - prev_dc)
30             prev_dc = int(zz[0])
31
32             # AC run-length
33             pairs = []
34             run = 0
35             for value in zz[1:]:
36                 if value == 0:
37                     run += 1
38                 else:
39                     while run > 15:
40                         pairs.append((15, 0))
41                         run -= 16
42                     pairs.append((run, int(value)))
43                     run = 0
44             if run > 0:
45                 pairs.append((0, 0))
46             ac_values.append(pairs)
47
48     return dc_values, ac_values
```

2.6 Huffman encoding

As the final step of the JPEG compression algorithm, the Huffman encoding, which is an entropy encoding algorithm, is used to further compress the data that need to be stored. In this step, frequently occurring values get encoded with fewer bits while less frequent values with a greater number of bits. It helps to reduce the average number of bits needed per symbol.

```
1 def size_category(value):
2     """
3     Number of bits needed for |value| (JPEG 'size' / category).
4     0 -> 0,   1 -> 1,   2..3 -> 2,   4..7 -> 3, ...
5     """
6     return int(abs(value)).bit_length()
```

```
7
8
9  def amplitude_bits(value, size):
10     """
11     Extra bits that store the exact value inside its category.
12     Negative values use one's complement (JPEG rule).
13     """
14     if size == 0:
15         return ""
16     if value < 0:
17         value = value + (1 << size) - 1
18     return format(value, f"0{size}b")
19
20
21 def build_huffman_table(symbols):
22     """
23     Build a Huffman code {symbol: bit string} from a list of symbols.
24     """
25     freq = Counter(symbols)
26
27     # Only one kind of symbol,  give it code "0"
28     if len(freq) == 1:
29         return {next(iter(freq)): "0"}
30
31     # heap items: [frequency, tie_breaker, [symbol, code], [symbol,
    code], ...]
32     heap = [[f, i, [sym, ""]] for i, (sym, f) in enumerate(freq.items()
    )]
33     heapq.heapify(heap)
34     counter = len(heap)
35
36     while len(heap) > 1:
37         low = heapq.heappop(heap)
38         high = heapq.heappop(heap)
39         for pair in low[2:]:
40             pair[1] = "0" + pair[1]
41         for pair in high[2:]:
42             pair[1] = "1" + pair[1]
43         heapq.heappush(heap, [low[0] + high[0], counter] + low[2:] +
    high[2:])
44         counter += 1
45
46     return {sym: code for sym, code in heap[0][2:]}
47
48
49 def dc_symbols(dc_values):
50     return [size_category(v) for v in dc_values]
51
52
53 def ac_symbols(ac_values):
54     # (run, size) ; EOB stays (0, 0) and ZRL stays (15, 0)
55     return [(run, size_category(v)) for block in ac_values for run, v
    in block]
56
57
58 def huffman_encode(dc_values, ac_values, dc_table, ac_table):
59     """
60     Encode one channel into a bit string.
```

```
    Per block: DC code + DC bits, then for each AC pair: AC code + AC
    bits.
    """
    bits = []
    for dc, block in zip(dc_values, ac_values):
        size = size_category(dc)
        bits.append(dc_table[size])
        bits.append(amplitude_bits(dc, size))

        for run, value in block:
            size = size_category(value)
            bits.append(ac_table[(run, size)])
            bits.append(amplitude_bits(value, size))
```

3 Code Repository

- Full Code available here : Link

4 References

1. Chroma Sub-Sampling : Link
2. How Image compression works : www.youtube.com
3. Images from Kodak Lossless True Color Image : Link
4. Baseline Quantization tables for JPEG Compression : Link
5. Run Length Encoding : Link
6. Huffman Encoding Link